

ESCOLA POLITÉCNICA DA UNIVERSIDADE DE SÃO PAULO



INTEGRANTES DO GRUPO	Nº USP
AUGUSTO MASSAYOSHI YOJO DE LIMA	12487121
GABRIEL ALMEIDA MATHIAS	14543284
LUCAS GALVÃO DE FRANÇA	4784490

PTC3527 - ANTEPROJETO DE FORMATURA EM TELECOMUNICAÇÕES RELATÓRIO DE FECHAMENTO ACADÊMICO DO ANTEPROJETO

Redução de Ruído Local em Tempo Real para Voz Humana: Comparação entre STFT e Wavelet com Avaliação de Viabilidade Embarcada

Orientador: Juan Luis Poletti Soto

14 de junho de 2026

Relatório de fechamento acadêmico da disciplina PTC3527, consolidando a evolução do benchmark algorítmico, a seleção perceptual do RNNoise, a integração com microfone virtual no Windows, os limites temporais encontrados no ambiente virtualizado e o protocolo de validação futura em Windows nativo.

SÃO PAULO

2026

Sumário

Índice de Ilustrações	3
1 Identificação do projeto	4
2 Contextualização da reformulação do escopo	4
3 Resumo do projeto reformulado	5
3.1 Objetivo geral	5
3.2 Objetivos específicos	6
4 Problema de pesquisa e hipótese de trabalho	6
5 Fundamentação teórica e matemática	7
5.1 Voz ruidosa e redução operacional de ruído	7
5.2 Abordagem por Fourier e STFT	7
5.3 Abordagem por Wavelet	8
5.4 Trade-offs esperados	9
6 Metodologia experimental proposta	9
6.1 Pipeline em ambiente computacional convencional	9
6.2 Amostras de voz, ruídos e níveis de SNR	10
6.3 Controle de variáveis	10
6.4 Refinamento com validação e conjunto final operacional	10
6.5 Conversão para estimador causal	11
6.6 Processamento reproduzível de arquivos em blocos	11
7 Métricas e análise de custo computacional	12
7.1 Métricas de qualidade e inteligibilidade	12
7.2 Tempo real, memória e latência	15
8 Referências bibliográficas e papel de cada referência	18
9 Atividades realizadas e resultados obtidos nesta fase	19
10 Fechamento acadêmico do protótipo integrado	24
10.1 Do baseline STFT ao benchmark de literatura	24
10.2 Arquitetura implementada	25
10.3 Gates isolados e prova funcional	25
10.4 Diagnóstico da cadência física e consumo WaveRT	26
10.5 Contrafactual SYSVAD versus HDA	26
10.6 Demonstração acadêmica em três camadas	27
10.7 Limitações metodológicas	27

10.8 Protocolo futuro em Windows nativo	27
11 Avaliação do andamento e justificativa dos desvios	28
12 Próximos passos técnicos	29
13 Questionário extensionista refinado	29
13.1 Público-alvo e plano de aplicação	29
13.2 Questionário proposto	30
14 Plano indicativo de continuidade	34
15 Riscos técnicos e estratégias de mitigação	34
16 Considerações finais	36
Referências	38

ÍNDICE DE ILUSTRAÇÕES

Figuras

Figura 1	Arquitetura do protótipo integrado. As medições distinguem fonte, DSP, transporte, escrita no driver e consumo do endpoint.	25
----------	---	----

Tabelas

Tabela 1	Comparação entre o escopo anterior e o escopo reformulado.	5
Tabela 2	Parâmetros usados no benchmark preliminar.	12
Tabela 3	Resultados médios preliminares por método e SNR alvo.	13
Tabela 4	Melhoria média de SNR com ruídos DEMAND	14
Tabela 5	Refinamento no conjunto final operacional	14
Tabela 6	Comparação do estimador causal	15
Tabela 7	Processamento causal de arquivo por tamanho de bloco	15
Tabela 8	Medições Windows input-only de 30 s, sem salvar WAV.	16
Tabela 9	Validação Windows prolongada da STFT causal	17
Tabela 10	Teste duplex curto no Windows com fone Bluetooth, sem salvar WAV. . .	17
Tabela 11	Full-duplex cabeado no Windows	18
Tabela 12	Função das referências no projeto reformulado.	19
Tabela 13	Resultado do split público final	24
Tabela 14	Resumo dos gates aceitos antes da análise temporal do endpoint.	26
Tabela 15	Contrafactual de notificações orientadas a evento na mesma VM.	26
Tabela 16	Sequência indicativa para validação nativa e publicação.	34
Tabela 17	Riscos técnicos do projeto e mitigação.	35
Tabela 18	Análise preliminar de viabilidade embarcada.	36

Gráficos

Gráfico 1	Melhoria média de SNR por método	22
Gráfico 2	Fator de tempo real médio por método	22
Gráfico 3	Formas de onda da condição representativa	23
Gráfico 4	Espectrogramas reduzidos da condição representativa	23

Códigos

1 Identificação do projeto

O projeto de formatura *Redução de Ruído Local em Tempo Real para Voz Humana: Comparação entre STFT e Wavelet com Avaliação de Viabilidade Embarcada* será desenvolvido na disciplina PTC3527 – Anteprojeto de Formatura em Telecomunicações, sob orientação do professor Juan Luis Poletti Soto, pelo grupo composto por Augusto Massayoshi Yojo de Lima, Gabriel Almeida Mathias e Lucas Galvão de França. O integrante Lucas Galvão de França foi incorporado ao grupo após a primeira apresentação, mantendo-se a continuidade do anteprojeto e ampliando a capacidade de execução técnica na etapa experimental.

A proposta situa-se em processamento digital de sinais, telecomunicações e sistemas embarcados, com foco específico em redução de ruído aplicada a voz humana. Diferentemente do escopo inicialmente apresentado, que tratava genericamente de sinais unidimensionais no tempo, esta versão delimita o problema para fala contaminada por ruído acústico e avalia, em primeiro lugar, técnicas implementáveis em ambiente computacional convencional. A execução em hardware é tratada como continuidade planejada para o próximo semestre, começando preferencialmente por Raspberry Pi, com ESP32 como etapa posterior dependente da complexidade final do pipeline. O Arduino Uno R3 é considerado apenas como limitação ou possibilidade de trabalho futuro, dadas suas restrições de memória, ponto flutuante, bibliotecas e custo de implementação.

2 Contextualização da reformulação do escopo

O tema original, *Revisão Sistemática da Literatura sobre Redução de Ruído de Sinais Unidimensionais no Tempo*, era tecnicamente relevante, mas amplo demais para orientar uma comparação experimental consistente. Ao incluir voz, áudio genérico, sensores, telemetria, sinais biomédicos, vibração e outras séries temporais, o projeto reunia classes de sinal com estatísticas, métricas, restrições e cenários de uso muito distintos. Essa amplitude dificultava tanto a definição de público-alvo quanto a construção de perguntas extensionistas úteis.

O feedback recebido sobre o questionário anterior apontou exatamente esse problema: a formatação e várias perguntas eram razoáveis, mas o público-alvo e o instrumento sofriam do mesmo excesso de abrangência do tema. A reformulação atual responde a essa crítica ao abandonar a generalidade de “qualquer sinal 1D” e concentrar a análise em voz humana, comunicação em tempo real, ruído acústico, privacidade, operação local, latência e viabilidade futura em hardware embarcado.

Essa mudança não representa ruptura desorganizada do anteprojeto. Trata-se de uma evolução técnica: o projeto preserva a base de processamento de sinais, a preocupação com métricas e a atividade extensionista, mas transforma a revisão bibliográfica em suporte para um protocolo experimental comparável.

Tabela 1: Comparação entre o escopo anterior e o escopo reformulado.

Dimensão	Escopo anterior	Escopo reformulado
Tipo de sinal	Sinais unidimensionais no tempo em geral.	Voz humana em sinais de áudio mono, com ruído acústico aditivo.
Produto técnico	Revisão sistemática e taxonomia ampla.	Pipeline experimental em PC para comparar STFT/Fourier e Wavelet, com análise de tempo real.
Público-alvo	Especialistas e usuários de muitos domínios.	Usuários de comunicação por voz, pessoas expostas a ambientes ruidosos e profissionais interessados em processamento local de áudio.
Questionário	Instrumento exploratório sobre ruído em sinais diversos.	Levantamento de requisitos para aplicações de voz: ruídos prioritários, latência, naturalidade, privacidade, operação offline, custo e energia.
Hardware	Possível consequência futura pouco delimitada.	Continuidade planejada: Raspberry Pi primeiro, ESP32 depois, Arduino Uno R3 apenas como limitação/trabalho futuro.

3 Resumo do projeto reformulado

O projeto reformulado tem como objetivo estudar, implementar e comparar técnicas de redução de ruído local para voz humana, com ênfase em operação de baixa latência e em futura viabilidade embarcada. Na etapa atual, será desenvolvido um pipeline em ambiente computacional convencional, como Windows ou Linux, capaz de carregar amostras de voz, selecionar ou sintetizar ruídos, gerar misturas em níveis controlados de SNR, aplicar métodos baseados em Transformada de Fourier de Curto Tempo (STFT) e Transformada Wavelet, reconstruir os sinais processados, calcular métricas objetivas e gerar gráficos comparativos.

O trabalho não busca prometer remoção completa de ruído nem protótipo embarcado integral nesta fase. O objetivo é construir uma comparação reprodutível entre abordagens de processamento de sinais, avaliar seus trade-offs de qualidade, inteligibilidade, custo computacional, uso de memória e latência, e indicar qual caminho é mais promissor para uma validação em hardware no semestre seguinte.

3.1 Objetivo geral

Desenvolver e validar, em ambiente computacional convencional, um pipeline de redução de ruído para voz humana que permita comparar métodos baseados em Fourier/STFT e Wavelet quanto à preservação da fala, supressão de ruído, custo computacional, latência e viabilidade de continuidade em sistemas embarcados.

3.2 Objetivos específicos

- a) reformular o problema de pesquisa para voz humana ruidosa, incorporando o feedback recebido sobre escopo e questionário;
- b) modelar o sinal observado como fala limpa contaminada por ruído e definir critérios operacionais de redução de ruído;
- c) implementar baselines baseados em STFT, como subtração espectral e ganho espectral inspirado em filtragem de Wiener;
- d) implementar uma abordagem por Transformada Wavelet Discreta, com decomposição multirresolução e limiarização de coeficientes;
- e) avaliar os métodos sob o mesmo conjunto de amostras de voz, ruídos, taxas de amostragem, durações e níveis de SNR;
- f) calcular métricas como SNR de entrada e saída, melhoria de SNR, MSE e, quando adequado, SI-SDR, STOI ou PESQ com as devidas ressalvas;
- g) medir tempo de processamento por quadro, fator de tempo real, latência algorítmica e uso aproximado de memória;
- h) preparar um cronograma realista para o segundo semestre, com benchmark algorítmico antes de integração embarcada.

4 Problema de pesquisa e hipótese de trabalho

O sinal observado será modelado, em tempo discreto, como

$$y[n] = x[n] + r[n], \quad (1)$$

em que $x[n]$ representa a fala limpa, $r[n]$ representa o ruído acústico e $y[n]$ é a fala ruidosa disponível ao sistema. Em aplicações reais, $x[n]$ não é observado diretamente durante a operação; ele é usado no protocolo experimental apenas quando há referência limpa para cálculo de métricas.

O problema de pesquisa pode ser formulado da seguinte maneira: sob restrições de baixa latência e custo computacional moderado, quais compromissos práticos aparecem ao comparar técnicas de redução de ruído de voz baseadas em STFT/Fourier e em Wavelet?

A hipótese de trabalho é que métodos baseados em STFT tendem a oferecer uma linha de base simples, interpretável e eficiente para ruídos aproximadamente estacionários ou com separação espectral razoável, enquanto métodos baseados em Wavelet podem ser promissores quando a estrutura da fala e do ruído aparece de modo mais localizado no tempo ou em escalas distintas. Essa hipótese será tratada como pergunta experimental, não como conclusão antecipada. O objetivo não é “eliminar todo ruído”, mas estimar $\hat{x}[n]$, uma versão processada que preserve inteligibilidade e naturalidade da fala com custo viável.

5 Fundamentação teórica e matemática

5.1 Voz ruidosa e redução operacional de ruído

A voz humana é um sinal não estacionário, com estrutura temporal, espectral e articulatória variável ao longo de uma frase. Vogais, consoantes, pausas, fricativas e transientes ocupam regiões diferentes no tempo e na frequência. Por isso, uma técnica de redução de ruído precisa evitar dois erros opostos: deixar ruído demais, reduzindo a inteligibilidade, ou suprimir componentes relevantes da fala, gerando voz artificial, abafada ou distorcida.

Neste projeto, a redução de ruído será definida operacionalmente como a construção de um estimador $\hat{x}[n] = \mathcal{D}\{y[n]\}$, em que $\mathcal{D}$ representa o algoritmo de denoising. A qualidade de $\hat{x}[n]$ será avaliada por métricas objetivas, inspeção de formas de onda, espectrogramas, escuta controlada pelos autores e análise de custo computacional. A métrica central não será uma pontuação isolada, mas o conjunto de trade-offs entre supressão de ruído, preservação da fala, latência e viabilidade de implementação.

5.2 Abordagem por Fourier e STFT

Como a fala varia ao longo do tempo, a Transformada de Fourier aplicada ao sinal inteiro perde a localização temporal das componentes espectrais. A STFT contorna essa limitação dividindo o sinal em quadros curtos, multiplicados por uma janela $w[n]$, com salto H e tamanho de FFT N . Para o quadro m e o bin de frequência k , define-se

$$Y[m, k] = \sum_{n=0}^{N-1} y[n + mH] w[n] e^{-j2\pi kn/N}. \quad (2)$$

Os parâmetros N , H , tipo de janela e sobreposição controlam o compromisso entre resolução temporal, resolução em frequência e latência. Janelas maiores melhoram a discriminação espectral, mas aumentam atraso e memória. Janelas menores reduzem latência, mas podem degradar a estimativa espectral. Para fala a 16 kHz, serão testadas janelas entre 20 ms e 32 ms, com sobreposição de 50% a 75%, por serem escolhas comuns em processamento de voz.

A linha de base espectral será implementada inicialmente por subtração espectral. Estima-se a magnitude média do ruído $|\hat{R}[m, k]|$, por exemplo a partir de trechos sem fala ou por média temporal suavizada, e calcula-se

$$|\hat{X}[m, k]| = \max \left(|Y[m, k]| - \alpha |\hat{R}[m, k]|, \beta |Y[m, k]| \right), \quad (3)$$

em que α controla a agressividade da supressão e β define um piso espectral para reduzir artefatos. Em uma variação por ganho espectral, aplica-se

$$\hat{X}[m, k] = G[m, k] Y[m, k], \quad (4)$$

com $G[m, k]$ inspirado em filtragem de Wiener, por exemplo $G = \xi/(1 + \xi)$, em que ξ é uma estimativa da SNR a priori no bin analisado. Inicialmente será preservada a fase do sinal ruidoso, aproximação comum em métodos clássicos de realce de fala.

A reconstrução temporal será feita por iSTFT e overlap-add:

$$\hat{x}[n] = \frac{\sum_m \hat{x}_m[n - mH] w[n - mH]}{\sum_m w^2[n - mH] + \varepsilon}, \quad (5)$$

em que ε evita divisão por zero em regiões de borda. Essa formulação torna a abordagem adequada a processamento por blocos e permite medir diretamente tempo por quadro e fator de tempo real.

5.3 Abordagem por Wavelet

Wavelets são relevantes para sinais não estacionários porque representam o sinal em diferentes escalas de tempo, em vez de usar apenas senos e cossenos globais. A Transformada Wavelet Discreta (DWT) decompõe o sinal em coeficientes de aproximação e detalhe. De forma conceitual,

$$a_{J,k} = \langle y, \phi_{J,k} \rangle, \quad d_{j,k} = \langle y, \psi_{j,k} \rangle, \quad (6)$$

em que $\phi_{J,k}$ representa funções de escala no nível J e $\psi_{j,k}$ representa wavelets em diferentes níveis j . Em implementação discreta, essa decomposição é obtida por bancos de filtros passa-baixas e passa-altas, seguidos de subamostragem.

A hipótese operacional do denoising por wavelets é que componentes estruturais da fala tendem a se concentrar em coeficientes relevantes, enquanto ruídos de menor estrutura podem se espalhar por coeficientes menores. Assim, aplica-se limiarização aos coeficientes de detalhe. Na limiarização dura,

$$\eta_H(d; \lambda) = \begin{cases} d, & |d| \geq \lambda, \\ 0, & |d| < \lambda, \end{cases} \quad (7)$$

enquanto na limiarização suave,

$$\eta_S(d; \lambda) = \text{sgn}(d) \max(|d| - \lambda, 0). \quad (8)$$

O limiar λ poderá ser definido por regra universal, por estimativa robusta do desvio padrão do ruído ou por ajuste experimental em validação. Serão avaliadas famílias como Daubechies, Symlets e Coiflets, com diferentes níveis de decomposição. A obra de Morettin sobre ondas e ondaletas será usada como base conceitual em português para amadurecer a relação entre análise de Fourier, análise de ondaletas e séries temporais; Mallat e Daubechies sustentam a formulação matemática mais geral da análise multirresolução.

5.4 Trade-offs esperados

A abordagem STFT tem como vantagem a maturidade de bibliotecas, a interpretação direta por espectrogramas, a relação clara com subtração espectral e filtragem de Wiener, e a integração natural com processamento por quadros. Seu risco principal é a criação de artefatos, especialmente ruído musical, além da dependência de estimativa adequada do ruído.

A abordagem Wavelet pode preservar estruturas localizadas e transientes, com custo computacional potencialmente baixo na DWT, mas depende fortemente da escolha da wavelet, do nível de decomposição, do limiar e do tratamento de bordas. Em fala, a vantagem não deve ser presumida: ela precisa aparecer nas métricas e na análise perceptual. A comparação proposta, portanto, busca identificar em que condições cada técnica é suficiente, promissora ou inadequada.

6 Metodologia experimental proposta

6.1 Pipeline em ambiente computacional convencional

Nesta etapa foi implementado um pipeline inicial em PC, em Python, usando NumPy, SciPy, PyWavelets, pandas e Matplotlib para cálculo, diagnóstico e geração de arquivos intermediários. A visualização final dos resultados no PDF, porém, passou a ser montada nativamente no LaTeX com `pgfplots` a partir de dados tabulados leves. A implementação ficou concentrada no pacote `benchmark_audio/`, com execução por linha de comando, sem depender de notebook. O comando principal de reprodução é:

```
python -m benchmark_audio.run_benchmark -prepare-demo-data
```

O pipeline implementado executa os seguintes passos:

1. baixa pequenas amostras públicas de fala humana do Free Spoken Digit Dataset, quando ainda não estão presentes localmente;
2. padroniza os sinais para áudio mono, 16 kHz e amplitude normalizada;
3. monta trechos demonstrativos de 3 s por concatenação de dígitos falados, com silêncio inicial de 0,30 s para estimativa simples de ruído;
4. gera ruídos controlados por script: branco, rosa, hum de 60/120 Hz e impulsivo;
5. mistura fala e ruído em SNRs alvo de -5 , 0, 5 e 10 dB;
6. aplica, sob as mesmas misturas, baseline ruidoso, STFT por subtração espectral, STFT com ganho inspirado em Wiener e Wavelet DWT com limiarização soft;
7. reconstrói os sinais processados e calcula SNR, melhoria de SNR, MSE, SI-SDR, tempo de processamento, RTF, latência algorítmica e memória aproximada;
8. gera automaticamente tabelas CSV, séries leves para `pgfplots`, figuras diagnósticas e exemplos curtos de áudio em `resultados/`.

O experimento sintético possui 5 trechos de fala, 4 tipos de ruído, 4 níveis de SNR e 4 métodos, totalizando 320 linhas de métricas em `resultados/tabelas/metricas_por_condicao.csv`.

Os resultados resumidos aparecem na Tabela 3. Na continuação, foi executada uma segunda matriz isolada com 12 trechos de ruído ambiental da base DEMAND, totalizando 960 linhas em `resultados/demand/tabelas/metricas_por_condicao.csv`. Após os benchmarks offline, também foi criado um protótipo de linha de comando em `realtime_audio/` para medir processamento por blocos no Windows. Essa segunda etapa não substitui o benchmark pareado de qualidade, pois não há referência limpa durante a captura real; sua função é verificar estabilidade, custo por bloco, latência reportada pelo sistema de áudio e funcionamento demonstrável do caminho captura-processa-reproduz.

6.2 Amostras de voz, ruídos e níveis de SNR

Sempre que possível, serão priorizadas bases públicas e bem documentadas, evitando coleta própria de voz nesta fase. Nesta implementação preliminar, a fala humana foi obtida do Free Spoken Digit Dataset, por ser uma base pública, leve e adequada para validar o pipeline sem baixar bases grandes. Não foram coletadas vozes próprias nem dados pessoais.

Na primeira rodada, os ruídos foram sintetizados por script para garantir reprodutibilidade e execução leve: ruído branco, ruído rosa, hum de rede elétrica e ruído impulsivo. Essa decisão permitiu validar mistura, processamento, métricas e geração de gráficos, mas limitava a validade externa.

Para ampliar o protocolo, foi criado o preparador `benchmark_audio/prepare_environmental_noise.py`. Foram selecionados quatro ambientes DEMAND a 16 kHz: cozinha (DKITCHEN), escritório (00FFICE), cafeteria (PCAFETER) e tráfego (STRAFFIC). De cada ambiente foram extraídos três segmentos não sobrepostos de 3 s do primeiro canal, totalizando 12 ruídos. Os arquivos brutos permaneceram fora do Git; origem, DOI, tamanho, MD5 e observação de licença foram registrados em manifesto. A rodada ambiental foi salva separadamente em `resultados/demand/`, evitando sobrescrever o benchmark sintético.

Os níveis de SNR de entrada usados foram -5 , 0 , 5 e 10 dB. Para cada condição, todos os métodos processaram exatamente a mesma mistura, evitando comparação enviesada.

6.3 Controle de variáveis

As comparações foram feitas por pares: cada amostra ruidosa foi processada por todos os métodos selecionados. Foram registrados semente de geração, normalização, amostra de voz, tipo de ruído, taxa de amostragem, SNR alvo e parâmetros dos algoritmos. A semente usada foi 3527. As rodadas sintética e DEMAND exploratória não tiveram separação formal; essa limitação foi corrigida no refinamento descrito a seguir.

6.4 Refinamento com validação e conjunto final operacional

Foi criado o executor `benchmark_audio/run_refinement.py`. Para não alterar retrospectivamente o benchmark histórico, os seis falantes usados nesta etapa foram preparados

em `dados/demo/clean_refinement/`. O prefixo fixo de 0,30 s foi removido antes das misturas, eliminando o silêncio inicial garantido.

A validação usou os falantes `jackson`, `nicolas` e `theo`, combinados com `DKITCHEN` e `OOFFICE`. O conjunto final operacional usou `george`, `lucas` e `yweweler`, combinados com `PCAFETER` e `STRAFFIC`. Cada divisão contém 72 condições: 3 falantes, 6 segmentos de ruído e 4 SNRs.

Foram avaliadas 144 configurações apenas na validação: 48 de subtração espectral, 24 Wiener e 72 Wavelet. Para STFT, foram comparadas janelas de 256/512 amostras, estimativa inicial e seleção offline de quadros de baixa energia, quantis, pisos e agressividades. Para Wavelet, foram testadas `db4`, `sym4` e `coif1`, níveis 3/5, limiares soft/hard, globais/por escala e fatores 0,50/0,75/1,00. Em cada família, a escolha maximizou a melhoria média de SNR na validação; SI-SDR, fração de degradação e RTF foram usados como desempate.

O conjunto final não foi usado pelo script para escolher parâmetros. Entretanto, ele não é historicamente cego, pois os quatro ambientes já haviam sido inspecionados na rodada DEMAND exploratória. Por isso, a expressão “conjunto final operacional” é mais adequada que “teste cego”.

6.5 Conversão para estimador causal

Após o refinamento offline, foi criado o núcleo `benchmark_audio/causal.py`, com estado explícito, reinicializável e independente de dispositivo. A API `process_block()` é a mesma usada pela captura Windows e pelo futuro processador de arquivos. Cada bloco de saída usa somente o estado acumulado até o bloco anterior; o espectro do bloco atual atualiza o estimador apenas para blocos posteriores.

Foram avaliadas 20 configurações adaptativas e uma calibração curta usando somente as 72 condições de validação. O estimador adaptativo combina quantil rolante por bin, quantil de energia global, atualização exponencial rápida em baixa energia e atualização lenta durante fala provável. A regra de seleção minimizou primeiro a fração de degradação de SNR e depois maximizou as melhorias médias de SNR e SI-SDR.

A configuração congelada usa $N = 512$, salto de 160 amostras, blocos de 20 ms, aquecimento de 250 ms, histórico causal de 500 ms, quantil espectral 0,22, quantil de energia 0,20, limiar de fala de 6 dB, fator exponencial 0,30 em baixa energia e 0,005 durante fala provável. A divisão final operacional foi processada somente depois dessa escolha; nenhuma gravação autoral futura foi usada.

6.6 Processamento reproduzível de arquivos em blocos

A etapa seguinte criou a CLI `realtime_audio/process_wav_blocks.py`, que processa WAVs sem dispositivo de áudio e instancia diretamente o mesmo `CausalSTFTProcessor` usado pela captura Windows. A entrada é convertida para mono a 16 kHz sem normalização automática; blocos de 10, 20 ou 32 ms são enviados em ordem, e o último bloco usa apenas as amostras

existentes. A saída preserva exatamente o número de amostras após a conversão e mantém alinhamento por índice, sem inserir silêncio para representar atraso.

Cada execução gera WAV PCM16, CSV por bloco e JSON com configuração, hashes SHA-256, formato original, picos, clipping, média, desvio, p50, p95, p99, pior caso, RTF e memória de estado. A estimativa de 32 ms associada à janela STFT é registrada separadamente: ela não representa deslocamento do arquivo, latência de driver, latência de dispositivo ou *round-trip* físico. O bypass é numericamente neutro antes da quantização, e a limitação à faixa $[-1, 1]$ ocorre somente na escrita PCM.

Para verificar o efeito operacional do particionamento sem reabrir a seleção de parâmetros, foi executada uma matriz com uma fala pública FSDD, um trecho de cafeteria do DEMAND, SNRs de -5 e 5 dB, os três tamanhos de bloco, bypass, subtração causal, Wiener causal e a referência offline de baixa energia. Também foram gerados vetores sintéticos curtos com configuração, resultados esperados, tolerâncias e SHA-256 para a futura transição ao Raspberry Pi.

Tabela 2: Parâmetros usados no benchmark preliminar.

Item	Valor usado nesta etapa
Taxa de amostragem	16 kHz.
Duração dos trechos	3 s por amostra, com normalização de amplitude antes da mistura.
SNR de entrada	-5 , 0 , 5 e 10 dB, calculados antes do processamento.
STFT	Janela Hann, $N = 512$, salto de 160 amostras, equivalente a 10 ms; subtração espectral com $\alpha = 1,2$ e piso 0,03; ganho Wiener simples com piso 0,05.
Wavelet	DWT com família db4, nível 5, limiarização soft e estimativa robusta de ruído por MAD.
Saídas	Benchmark sintético em <code>resultados/</code> ; rodada ambiental em <code>resultados/demand/</code> ; refinamento em <code>resultados/demand_refinement/</code> ; estimador causal em <code>resultados/causal_estimator/</code> ; arquivos em blocos em <code>resultados/file_blocks/</code> ; CSVs, JSONs, figuras, áudios curtos e logs de tempo real.

7 Métricas e análise de custo computacional

7.1 Métricas de qualidade e inteligibilidade

As métricas centrais desta etapa foram simples e reproduzíveis. A SNR de entrada ou saída foi calculada, quando havia referência limpa, por

$$\text{SNR} = 10 \log_{10} \frac{\sum_n x^2[n]}{\sum_n (x[n] - \hat{x}[n])^2}. \quad (9)$$

A melhoria de SNR foi calculada por

$$\Delta\text{SNR} = \text{SNR}_{\text{out}} - \text{SNR}_{\text{in}}. \quad (10)$$

Também foi registrado o erro quadrático médio,

$$\text{MSE} = \frac{1}{N} \sum_{n=0}^{N-1} (x[n] - \hat{x}[n])^2. \quad (11)$$

Também foi calculada a SI-SDR, por ser menos dependente de escala global. Para uma estimativa $\hat{x}$, define-se uma projeção $s_{\text{alvo}} = \alpha x$, com $\alpha = \langle \hat{x}, x \rangle / \|x\|^2$, e calcula-se

$$\text{SI-SDR} = 10 \log_{10} \frac{\|s_{\text{alvo}}\|^2}{\|\hat{x} - s_{\text{alvo}}\|^2}. \quad (12)$$

STOI não foi usado no primeiro benchmark, mas passou a integrar o harness de literatura com a versão `pystoi 0.4.1`. A divergência entre SNR, SI-SDR, STOI, preservação de envelope e percepção foi mantida visível: nenhuma métrica isolada determinou a seleção. PESQ não foi incorporado, em razão das restrições práticas de uso e licença. A decisão final combinou métricas pareadas, custo, causalidade, memória, latência e escuta controlada.

Tabela 3: Resultados médios preliminares por método e SNR alvo.

SNR alvo (dB)	Método	SNR saída (dB)	Melhoria (dB)	SI-SDR (dB)	RTF
-5	Ruidoso	-5,00	0,00	-5,00	0,0000
-5	STFT Wiener	2,46	7,46	1,37	0,0012
-5	STFT subtração	4,63	9,63	2,96	0,0021
-5	Wavelet soft	-2,85	2,15	-7,23	0,0003
0	Ruidoso	0,00	0,00	-0,01	0,0000
0	STFT Wiener	6,89	6,89	6,38	0,0012
0	STFT subtração	8,42	8,42	7,76	0,0020
0	Wavelet soft	1,16	1,16	-1,13	0,0003
5	Ruidoso	5,00	0,00	4,99	0,0000
5	STFT Wiener	11,22	6,22	11,01	0,0012
5	STFT subtração	12,15	7,15	12,06	0,0020
5	Wavelet soft	5,15	0,15	4,08	0,0003
10	Ruidoso	10,00	0,00	9,99	0,0000
10	STFT Wiener	15,49	5,49	15,44	0,0012
10	STFT subtração	15,89	5,89	16,12	0,0021
10	Wavelet soft	9,08	-0,92	8,71	0,0003

Tabela 4: Melhoria média de SNR na rodada ambiental DEMAND, agregada sobre 5 trechos de fala e 12 segmentos de ruído. Fonte: `resultados/demand/tabelas/resumo_por_metodo_snr.csv`.

SNR alvo (dB)	STFT subtração (dB)	STFT Wiener (dB)	Wavelet soft (dB)
-5	7,69	4,62	0,28
0	7,31	4,50	0,14
5	6,82	4,33	-0,05
10	6,21	4,07	-0,38

Na rodada DEMAND, a subtração espectral manteve a maior melhoria média de SNR, seguida pelo ganho Wiener. Entretanto, a magnitude variou entre ambientes. Agregando os quatro níveis de SNR, a melhoria da subtração espectral foi aproximadamente 10,00 dB em cozinha, 8,45 dB em escritório, 4,11 dB em cafeteria e 5,47 dB em tráfego. A Wavelet soft ficou próxima de zero no agregado e alternou pequenos ganhos e perdas. Esses resultados reforçam apenas o comportamento dos parâmetros atuais; não demonstram superioridade geral de STFT, pois os segmentos pertencem a um único canal por ambiente, a fala ainda contém silêncio inicial favorável à estimativa de ruído e não houve separação entre validação e teste final.

Tabela 5: Comparação agregada no conjunto final operacional, sem silêncio inicial garantido. Degradação: fração de condições com melhoria de SNR negativa. Fonte: elaboração própria.

Configuração	Δ SNR (dB)	Δ SI-SDR (dB)	Degradação	RTF
STFT subtração, estimativa inicial	1,82	-0,16	33,3%	0,00424
STFT subtração, baixa energia	4,85	3,72	0,0%	0,00426
STFT Wiener, estimativa inicial	1,30	-0,40	27,8%	0,00232
STFT Wiener, baixa energia	2,92	2,25	0,0%	0,00238
Wavelet db4, nível 5, soft	0,32	-0,46	19,4%	0,00087
Wavelet sym4, nível 3, hard	0,03	0,01	11,1%	0,00058

Na validação, foram escolhidos $N = 512$, salto 160, quantil de baixa energia 0,35, $\alpha = 1,5$ e piso 0,02 para subtração espectral; o Wiener manteve $N = 512$, salto 160, quantil 0,35 e piso 0,05. A Wavelet escolhida foi `sym4`, nível 3, limiar hard global e fator 0,50.

Sem silêncio inicial, a estimativa pelos primeiros 0,25 s perdeu robustez, principalmente em SNR de entrada alta: no conjunto final, a subtração padrão piorou a SNR média em 2,45 dB nas condições de 10 dB. A seleção offline de quadros de menor energia eliminou degradações de SNR nas 72 condições finais e melhorou também a SI-SDR. A Wavelet refinada reduziu a fração de degradação e neutralizou a perda média de SI-SDR, mas sua melhoria de SNR ficou praticamente nula. A estimativa de baixa energia continua sendo uma referência não causal; sua aproximação causal é comparada a seguir.

Tabela 6: Comparação no conjunto final operacional. O modo de baixa energia examina o arquivo completo e funciona apenas como limite superior operacional. Fonte: elaboração própria.

Configuração	Δ SNR (dB)	Δ SI-SDR (dB)	Degradação
Bypass	0,00	0,00	0,0%
Subtração, calibração causal	0,96	-2,38	33,3%
Subtração, estimativa inicial antiga	1,82	-0,16	33,3%
Subtração, adaptativa causal	3,76	2,65	0,0%
Subtração, baixa energia offline	4,85	3,72	0,0%
Wiener, adaptativo causal	1,68	1,35	0,0%
Wiener, baixa energia offline	2,92	2,25	0,0%

A subtração causal recuperou a maior parte do ganho do estimador offline e não degradou nenhuma das 72 condições finais. A calibração curta foi instável sem silêncio inicial, com perda média de SI-SDR e degradação em um terço das condições. O Wiener causal também evitou degradações, mas ficou abaixo da subtração. O estado persistente máximo medido foi 60.900 bytes, aproximadamente 59,5 KiB, sem contar o interpretador Python e temporários internos das bibliotecas.

Tabela 7: Matriz PC-2 com uma fala FSDD, ruído DEMAND de cafeteria e SNRs de -5 e 5 dB. Tempos são de arquivo em blocos e não incluem dispositivo. Fonte: resultados/file_blocks/tabelas/comparison_summary.csv.

Método	Bloco (ms)	Δ SNR (dB)	Δ SI-SDR (dB)	RTF	p99 (ms)
Subtração causal	10	3,36	1,22	0,111	2,12
Subtração causal	20	3,27	1,27	0,081	2,76
Subtração causal	32	3,25	1,35	0,068	3,41
Wiener causal	10	1,54	0,74	0,107	2,49
Wiener causal	20	1,33	0,67	0,078	2,40
Wiener causal	32	1,27	0,68	0,069	3,66

Na mesma amostra, as referências não causais de baixa energia obtiveram 4,31 dB de melhoria de SNR e 2,14 dB de SI-SDR para subtração, e 2,85 dB e 1,68 dB para Wiener. Todos os comprimentos foram preservados, o deslocamento por índice foi zero, nenhum bloco excedeu seu orçamento e o estado causal máximo permaneceu em 60.900 bytes. Blocos de 10 ms tiveram leve vantagem de SNR; 32 ms reduziram o RTF e tiveram leve vantagem de SI-SDR na subtração. A matriz é pequena e serve para equivalência e operação, não para nova escolha algorítmica.

7.2 Tempo real, memória e latência

A viabilidade para tempo real foi avaliada inicialmente por:

$$\text{RTF} = \frac{T_{\text{proc}}}{T_{\text{audio}}}, \quad (13)$$

em que $\text{RTF} < 1$ indica processamento mais rápido que o tempo do áudio. No benchmark offline foram registrados tempo total por arquivo, RTF médio por método, latência algorítmica estimada e memória aproximada. A CLI de arquivo em blocos mede a mesma chamada causal sem driver e não deve ser interpretada como captura física. Na continuação da fase, o protótipo

em `realtime_audio/windows_realtime.py` passou a medir cada bloco de áudio capturado pelo Windows, registrando média, pior caso, desvio padrão, RTF por bloco e eventos reportados pelo *stream*.

A latência algorítmica foi estimada a partir do tamanho da janela, do salto e do tipo de método. Para STFT causal por blocos, janelas de 20 ms a 32 ms já impõem um piso de atraso associado ao enquadramento. Para Wavelet em blocos, a latência dependerá do tamanho do bloco, do tratamento de bordas e da necessidade de amostras futuras. A meta inicial continua sendo manter atraso total abaixo de 100 ms em PC e discutir a possibilidade de meta mais exigente, como 50 ms, apenas se os resultados forem compatíveis.

O uso de memória foi aproximado pelo tamanho dos buffers de entrada e saída, matrizes de coeficientes, espectrogramas temporários, parâmetros dos algoritmos e estruturas auxiliares. No benchmark preliminar em PC, todos os métodos rodaram muito abaixo do tempo real: a STFT por subtração espectral apresentou RTF médio de aproximadamente 0,0021, a STFT Wiener aproximadamente 0,0012 e a Wavelet soft aproximadamente 0,00032. Esses valores indicam folga em PC, mas não devem ser transferidos diretamente para microcontroladores, pois bibliotecas vetorizadas e memória abundante mascaram custos de portabilidade.

No processamento causal em blocos de 20 ms, a subtração apresentou RTF médio 0,068 e p99 médio de 3,08 ms no conjunto final operacional; o pior bloco foi 13,31 ms. Na execução em lote da validação apareceu um pico isolado de 104,12 ms, embora o p99 médio tenha sido 3,74 ms e o RTF médio 0,078. Esse pico é registrado como jitter do processo Python/sistema e motivou uma validação Windows prolongada do núcleo causal congelado. Em autoteste sintético posterior de 1 s, a média foi 1,58 ms, o p99 4,01 ms e o pior bloco 4,62 ms, sem exceder o orçamento de 20 ms.

Em captura real sem reprodução, foram executados testes de 30 s no Windows, com áudio mono a 16 kHz, blocos de 20 ms e sem salvamento de WAV de voz. A entrada padrão foi `SteelSeries Sonar - Microphone` via MME. A Tabela 8 resume apenas as linhas de 30 s da tabela `resultados/tabelas/realtime_windows_input_only.csv`; duas medições curtas anteriores de 3 s permaneceram no CSV apenas como histórico de validação.

Tabela 8: Medições Windows input-only de 30 s, sem salvar WAV.

Método	Blocos	Média (ms)	Pior (ms)	Desvio (ms)	RTF médio	Lat. total (ms)
bypass	1498	0,031	0,134	0,024	0,00157	40
stft_subtraction	1498	0,471	1,929	0,175	0,0236	72
stft_wiener	1498	0,412	1,423	0,175	0,0206	72
wavelet_soft	1498	0,258	14,749	0,394	0,0129	60

Nos quatro métodos da Tabela 8, `status_counts` ficou vazio, isto é, não houve evento de erro de captura reportado pelo *stream* durante os 30 s. O pior caso de `wavelet_soft` foi bem mais alto que sua média, mas ainda inferior ao orçamento de 20 ms por bloco. Durante essa rodada, o PyWavelets emitiu um aviso numérico de limiarização soft; a checagem dos CSVs por bloco não encontrou NaN nem infinito, mas o aviso deve ser investigado antes de conclusões

finais sobre Wavelet em streaming.

Após a decisão técnica, a implementação PC escolhida foi exercitada em rodadas de 600 s com `stft_subtraction`, `noise-mode adaptive`, bloco de 20 ms e parâmetros congelados. A Tabela 9 resume os artefatos de `resultados/windows_realtime_longrun/`. A primeira rodada foi sintética, sem dispositivo físico; a segunda usou o Microfone (USB Audio Device) via MME, em modo `-input-only` e sem salvar áudio de microfone.

Tabela 9: Validação Windows prolongada da subtração STFT causal adaptativa em rodadas de 600 s. Em ambas, nenhum bloco excedeu o orçamento externo de 20 ms e `status_counts` ficou vazio. Fonte: JSONs de `resultados/windows_realtime_longrun/`.

Rodada	Blocos	Média (ms)	p95 (ms)	p99 (ms)	Pior (ms)	RTF médio	> 20 ms
Self-test sintético	30000	0,987	1,271	1,594	4,127	0,049	0
Input-only USB/MME	29998	1,280	2,205	3,904	6,799	0,064	0

A rodada prolongada resolve a pendência operacional do pico isolado observado em arquivo: no PC testado, o núcleo STFT causal sustentou dez minutos sintéticos e dez minutos de captura física de entrada com folga em relação ao orçamento por bloco. A latência total de 72 ms registrada no JSON da rodada física soma 32 ms de latência algorítmica estimada e 40 ms de entrada reportada pelo driver; esse valor não é *round-trip* físico, pois a validação foi `input-only` e não mediu reprodução, loopback ou latência de saída. O Bluetooth não foi usado como evidência de baixa latência.

Também foi executado um teste duplex curto, isto é, com captura, processamento e reprodução local. A saída foi fixada no fone Bluetooth HUAWEI FreeBuds SE 2, índice 7 via MME, e a entrada no índice 1. O teste durou 5 s por método, sem salvar WAV. A Tabela 10 resume `resultados/tabelas/realtime_windows_duplex.csv`.

Tabela 10: Teste duplex curto no Windows com fone Bluetooth, sem salvar WAV.

Método	Blocos	Média (ms)	Pior (ms)	RTF médio	Lat. total (ms)
bypass	248	0,032	0,107	0,00162	240
stft_subtraction	248	0,472	1,463	0,0236	272

O teste duplex curto também terminou sem eventos em `status_counts`. O usuário repetiu a captura localmente com o fone Bluetooth e informou que a execução ocorreu bem, sem relato de eco, realimentação ou desconforto. Ainda assim, a latência total estimada ficou dominada pela saída Bluetooth, que reportou 200 ms; portanto, o resultado deve ser lido como validação funcional curta da CLI, não como demonstração de baixa latência perceptual. Uma demonstração mais exigente deve usar saída cabeada ou outro dispositivo de menor atraso e, idealmente, medição física de *round-trip*.

Essa demonstração mais exigente foi então repetida com fone cabeado/controlado, usando Microfone (USB Audio Device), MME no índice 2 e Alto-falantes (AB13X USB Audio), MME no índice 8. O usuário manteve o volume em 20/100 e confirmou retorno audível no

fone cabeado, sem eco ou desconforto nas rodadas curtas. A Tabela 11 resume as execuções válidas com `stft_subtraction`; tentativas com WASAPI e WDM-KS, embora tivessem menor latência declarada, não abriram *stream* duplex a 16 kHz neste PC.

Tabela 11: Full-duplex cabeado/controlado no Windows com a STFT causal adaptativa congelada, sem salvar WAV. Fonte: `resultados/tabelas/realtime_windows_wired.csv`.

Rodada	Blocos	Média (ms)	p99 (ms)	Pior (ms)	RTF médio	> 20 ms
STFT cabeada, 5 s	248	1,265	3,284	3,941	0,063	0
STFT cabeada, 30 s	1498	1,239	2,688	4,811	0,062	0
STFT cabeada, 600 s	29998	1,259	3,283	7,301	0,063	0

A rodada cabeada de 600 s terminou com `status_counts` vazio, estado máximo de 60.900 bytes e nenhum bloco acima de 20 ms. Portanto, ela substitui a demonstração Bluetooth como evidência funcional de captura-processamento-reprodução com saída controlada. Ainda assim, a saída MME cabeada reportou 200 ms de latência de saída; o total estimado de 272 ms no JSON soma essa latência de I/O reportada e 32 ms algorítmicos, não sendo uma medição física de *round-trip*.

A análise de hardware nesta fase aponta Raspberry Pi como a primeira plataforma embarcada plausível, por manter Linux, Python/C++ e memória suficiente para depuração. ESP32/ESP32-S3 deve ser considerado apenas depois de simplificar buffers, dependências e representação numérica. O Arduino Uno R3 segue como referência de limitação severa: 2 KB de SRAM, 32 KB de flash, ausência de FPU e falta de interface de áudio adequada tornam STFT/Wavelet de voz em tempo real impraticável sem simplificações extremas ou hardware externo.

8 Referências bibliográficas e papel de cada referência

A bibliografia será usada como suporte de decisões técnicas, não como lista decorativa. Oppenheim e Schafer, Proakis e Manolakis fundamentam amostragem, DFT, FFT, janelamento e processamento digital discreto. Boll justifica a subtração espectral como baseline clássico. Ephraim e Malah sustentam a formulação estatística por amplitude espectral de curto termo. Loizou organiza a área de realce de fala, métricas e trade-offs perceptuais. Morettin, Mallat e Daubechies sustentam a análise por ondaletas e multirresolução. Donoho e Donoho–Johnstone sustentam a limiarização wavelet. Taal et al. e ITU-T P.862 fundamentam a discussão sobre métricas perceptuais, com ressalvas de uso. VoiceBank–DEMAND, DEMAND, LibriSpeech e FSDD apoiam a construção de protocolo experimental reprodutível. NumPy, SciPy, PyWavelets e Matplotlib documentam as ferramentas usadas no pipeline preliminar; `pgfplots` fica como camada de visualização vetorial no documento final.

Tabela 12: Função das referências no projeto reformulado.

Referência	Papel no relatório e na metodologia
Oppenheim e Schafer; Proakis e Manolakis	Fundamentam processamento discreto, DFT, FFT, janelamento, filtragem e reconstrução.
Boll; Ephraim e Malah; Loizou	Sustentam métodos espectrais clássicos de realce de fala, incluindo subtração espectral, estimadores MMSE e discussão de artefatos.
Morettin; Mallat; Daubechies	Organizam a passagem de Fourier para ondaletas, análise multirresolução, escolha de bases e interpretação de coeficientes tempo-escala.
Donoho; Donoho e Johnstone	Justificam denoising por limiarização hard/soft e shrinkage em coeficientes wavelet.
Taal et al.; ITU-T P.862	Orientam o uso cauteloso de STOI e PESQ como métricas perceptuais dependentes de referência limpa.
VoiceBank– DEMAND, DEMAND e LibriSpeech	Apoiam a seleção de fala limpa e ruídos ambientais para misturas controladas e comparação reprodutível.
FSDD	Fornece amostras públicas pequenas de fala humana para validação leve do pipeline demonstrativo.
NumPy, SciPy, PyWavelets, Matplotlib e pgfplots	Sustentam a implementação computacional reprodutível, incluindo vetorização, STFT/iSTFT, DWT, geração de dados intermediários e visualização vetorial no LaTeX.

9 Atividades realizadas e resultados obtidos nesta fase

Nesta fase, além da reformulação técnica do anteprojeto, foi implementado e executado um benchmark preliminar em PC. As atividades realizadas foram:

- análise crítica da entrega anterior, identificando elementos a preservar: estrutura acadêmica, preocupação com métricas, cronograma, referências clássicas e atividade extensionista;
- leitura da reformulação do tema, do questionário extensionista já ajustado e do dossiê técnico, filtrando o que era específico de PIBITI;
- incorporação do feedback do professor, com redução do escopo para voz humana e comunicação em tempo real;
- redefinição do problema de pesquisa em torno do modelo $y[n] = x[n] + r[n]$;
- seleção da comparação central do projeto: Fourier/STFT versus Wavelet;
- definição de protocolo experimental inicial com amostras de voz, ruídos, níveis de SNR, métricas e análise de custo computacional;
- refinamento do questionário extensionista para que cada pergunta gere informação acionável;
- reorganização do cronograma para priorizar benchmark algorítmico em PC antes de qualquer integração embarcada;
- criação do pacote `benchmark_audio/`, com script de execução por linha de comando;
- implementação de leitura e padronização de WAV, geração de ruídos, mistura por SNR,

- reconstrução e cálculo de métricas;
- k) implementação dos baselines ruidoso, STFT por subtração espectral, STFT com ganho inspirado em Wiener e Wavelet DWT com limiarização soft;
 - l) geração de tabelas, séries `pgfplots`, figuras diagnósticas e áudios curtos de demonstração em `resultados/`;
 - m) criação do preparador DEMAND, com manifesto de origem, DOI, licença, tamanhos e MD5, mantendo os arquivos brutos fora do Git;
 - n) execução de uma rodada ambiental isolada com 12 segmentos de cozinha, escritório, cafeteria e tráfego, gerando 960 linhas de métricas em `resultados/demand/`;
 - o) ampliação da fala para seis falantes FSDD no refinamento, preservando o conjunto histórico de cinco falantes em pasta separada;
 - p) criação de divisão entre validação e conjunto final operacional, com falantes e ambientes distintos;
 - q) avaliação de 144 configurações STFT/Wavelet sem silêncio inicial garantido e seleção de parâmetros apenas na validação;
 - r) separação de um núcleo reutilizável em `benchmark_audio/denoise.py`, usado tanto pelo benchmark offline quanto pelo protótipo em tempo real;
 - s) implementação de estimador causal em `benchmark_audio/causal.py`, com quantil rolante por bin, decisão de baixa energia, atualização lenta durante fala e estado reinicializável;
 - t) avaliação de 20 configurações causais adaptativas e uma calibração curta somente na validação, seguida de comparação congelada no conjunto final operacional;
 - u) criação da CLI de WAV em blocos, com conversão mono/16 kHz sem normalização, preservação de comprimento, CSV por bloco, JSON por execução e recusa de sobrescrita silenciosa;
 - v) execução da matriz PC-2 em 10, 20 e 32 ms e geração de vetores sintéticos versionáveis com resultados esperados e SHA-256;
 - w) criação da CLI `realtime_audio/windows_realtime.py`, com autoteste sintético, modo seguro `-input-only`, medição por bloco e teste duplex curto;
 - x) execução de medições reais no Windows: quatro métodos em input-only por 30 s, dois métodos em duplex curto com fone Bluetooth, validação prolongada de 600 s da STFT causal adaptativa em self-test e input-only, e demonstração full-duplex cabeada de 600 s;
 - y) registro de diário técnico, auditoria crítica dos resultados e checkpoints versionados em Git.

Os resultados obtidos ainda são preliminares, mas já deixam de ser apenas metodológicos. O experimento sintético usou 5 trechos de fala humana do FSDD, 4 ruídos sintetizados, SNRs de -5 , 0 , 5 e 10 dB e 4 métodos. O baseline ruidoso confirmou melhoria de SNR igual a zero, como controle. A STFT por subtração espectral apresentou a maior melhoria média de SNR no arranjo testado, entre $5,89$ dB e $9,63$ dB. A STFT Wiener também melhorou a SNR, entre $5,49$ dB e $7,46$ dB. A Wavelet soft foi mais rápida, mas teve ganho pequeno e inconsistente, inclusive piorando a SNR média em $0,92$ dB no caso de SNR alvo de 10 dB.

Na rodada ambiental DEMAND, a STFT por subtração espectral obteve melhoria média entre 6,21 dB e 7,69 dB conforme a SNR alvo, enquanto a STFT Wiener ficou entre 4,07 dB e 4,62 dB. A Wavelet soft variou de ganho de 0,28 dB a perda de 0,38 dB. A dispersão por ambiente foi relevante: cozinha e escritório favoreceram mais os baselines STFT que cafeteria e tráfego. Portanto, a nova rodada aumenta a validade externa em relação aos ruídos sintetizados, mas também mostra que o tipo de ruído altera substancialmente a magnitude do ganho.

No refinamento sem silêncio inicial, os parâmetros padrão de STFT perderam desempenho e apresentaram melhoria negativa de SI-SDR no agregado final. A estimativa offline por quadros de baixa energia elevou a melhoria média para 4,85 dB na subtração e 2,92 dB no Wiener, sem degradação de SNR nas 72 condições finais. Entre 72 configurações Wavelet, `sym4`, nível 3 e limiar hard reduzido foi a mais equilibrada na validação; no conjunto final, porém, permaneceu praticamente neutra em SNR e SI-SDR. O resultado indica que o refinamento reduziu dano, mas não aproximou a Wavelet da supressão obtida pelas STFTs.

Na etapa causal, a subtração adaptativa obteve 3,76 dB de melhoria de SNR e 2,65 dB de SI-SDR no conjunto final operacional, sem degradações. A calibração causal curta ficou em 0,96 dB de SNR, perdeu 2,38 dB de SI-SDR e degradou um terço das condições. O Wiener causal atingiu 1,68 dB e 1,35 dB, respectivamente, também sem degradações. Assim, a aproximação causal reduziu a distância para o limite offline sem depender de silêncio inicial fixo.

No processamento reproduzível de arquivo, o mesmo núcleo preservou comprimento e alinhamento nos três tamanhos de bloco. A subtração causal variou de 3,25 a 3,36 dB de melhoria de SNR, e o Wiener de 1,27 a 1,54 dB. Nenhum bloco excedeu o orçamento. A execução CLI versionada reproduziu exatamente o SHA-256 do vetor causal esperado, confirmando equivalência entre helper, CLI e núcleo.

As medições em tempo real acrescentam uma conclusão de natureza diferente: elas não medem melhoria de SNR, mas indicam que o custo computacional por bloco ainda fica com folga no notebook Windows. Em input-only de 30 s, os métodos STFT e Wavelet permaneceram abaixo de RTF 1 em todos os blocos observados, sem eventos de erro de captura reportados. Depois, a STFT causal adaptativa congelada sustentou 600 s em self-test e 600 s de captura física input-only, com zero blocos acima de 20 ms; no caso físico, o pior bloco foi 6,799 ms e o RTF médio foi 0,064. No caminho full-duplex cabeado, a mesma STFT sustentou 600 s com pior bloco de 7,301 ms, RTF médio de 0,063, zero blocos acima de 20 ms e `status_counts` vazio. A limitação principal observada deixou de ser Bluetooth, mas a saída MME cabeada ainda reportou 200 ms de latência de saída, logo a rodada não é prova de baixa latência física ponta a ponta.

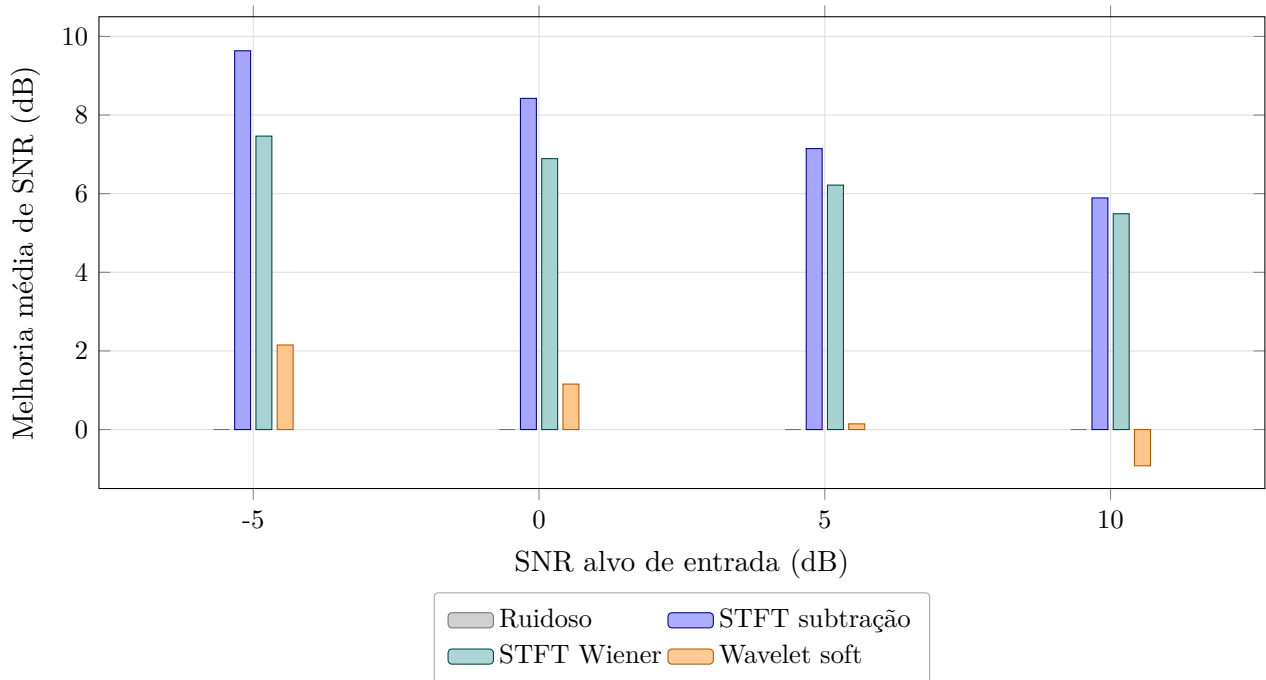


Gráfico 1: Melhoria média de SNR por método no benchmark preliminar. Os dados vêm de `resultados/tabelas/resumo_por_metodo_snr.csv` e foram exportados por `benchmark_audio/run_benchmark.py` para `resultados/pgfplots/melhoria_snr.csv`. O resultado é preliminar e usa fala FSDD, ruídos sintetizados e SNRs alvo de -5 , 0 , 5 e 10 dB.

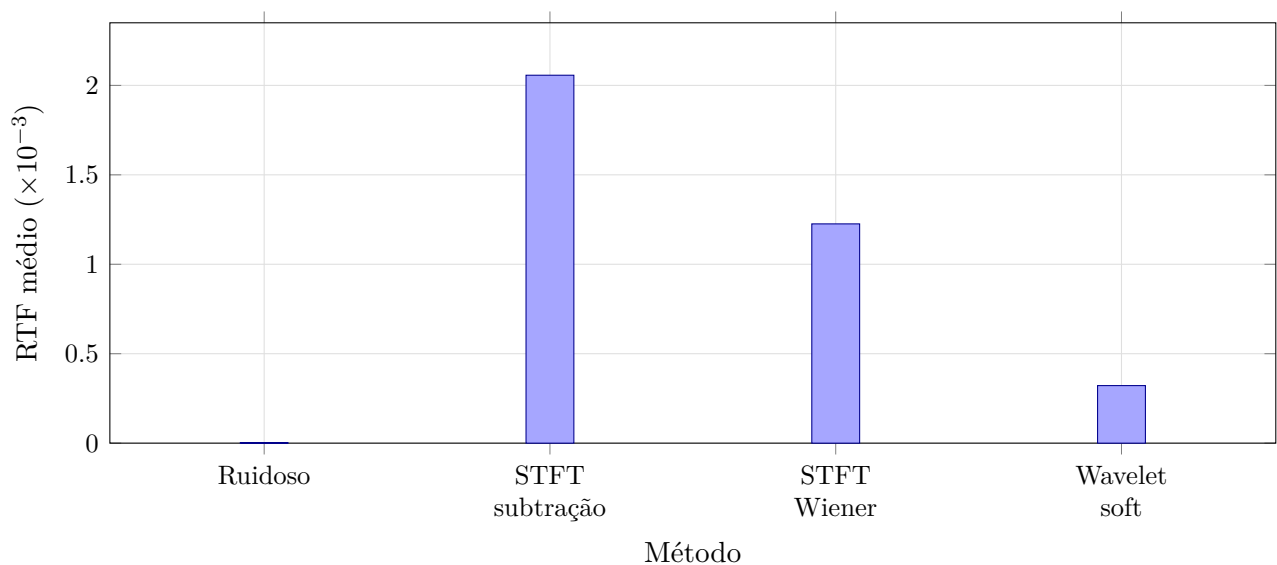


Gráfico 2: Fator de tempo real médio por método, agregado sobre os SNRs alvo do benchmark preliminar. O eixo vertical mostra $1000 \times \text{RTF}$, com dados exportados pelo script para `resultados/pgfplots/rtf_por_metodo.csv`; valores abaixo de 1 em RTF indicam processamento mais rápido que a duração do áudio, mas a medição ainda é em PC e não equivale à latência embarcada por blocos.

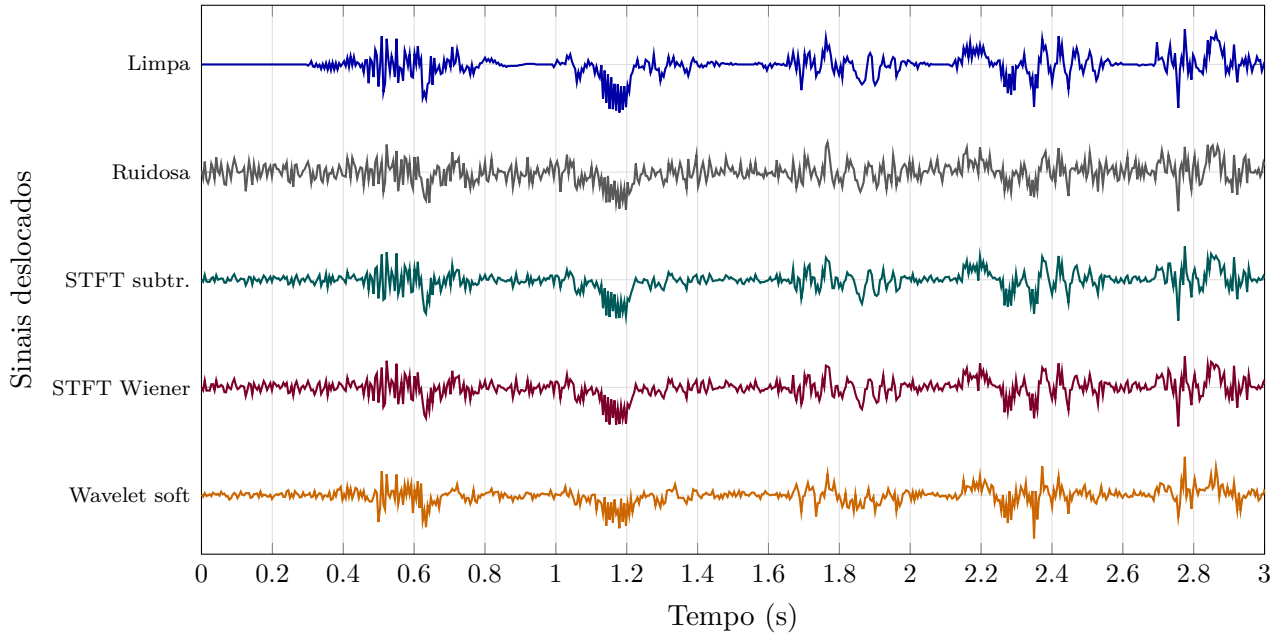


Gráfico 3: Formas de onda de uma condição representativa do benchmark preliminar, decimadas para manter o LaTeX leve. As séries foram exportadas por `benchmark_audio/run_benchmark.py` para `resultados/pgfplots/formas_onda_exemplo.csv`; os deslocamentos verticais servem apenas para separar visualmente os sinais.

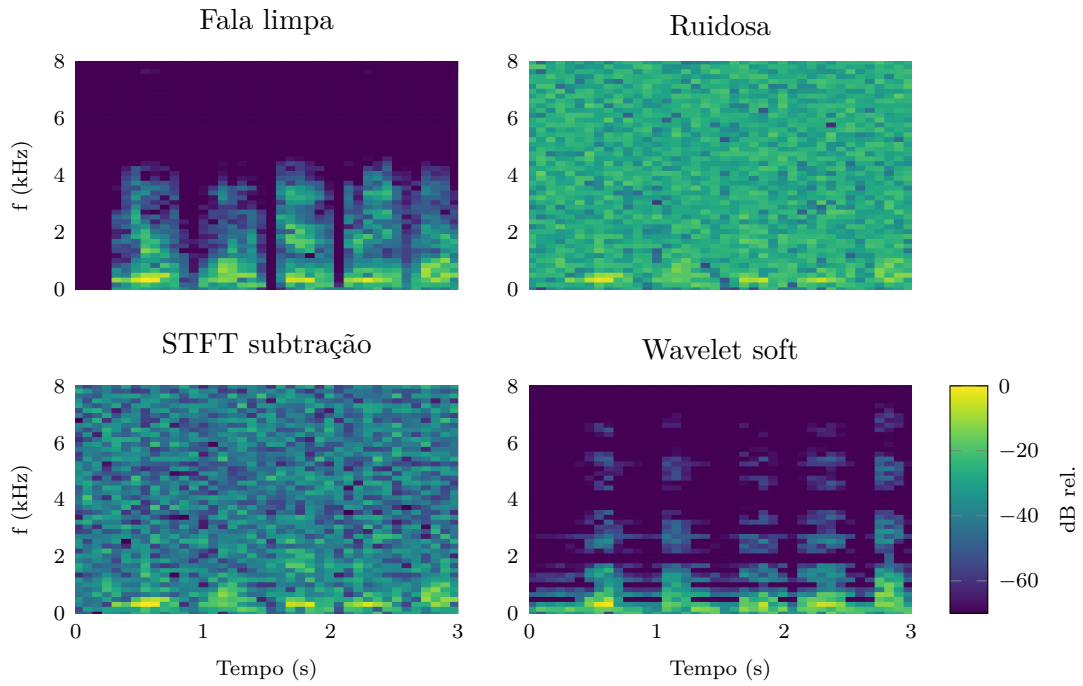


Gráfico 4: Espectrogramas reduzidos da mesma condição representativa do Gráfico 3. As matrizes foram exportadas em dB relativo ao pico global do exemplo para `resultados/pgfplots/espectrograma_*.csv`, com 48 frequências por 40 instantes; a redução da malha é uma escolha deliberada para manter a compilação confortável.

Os arquivos de áudio de demonstração foram salvos em `resultados/audio/`. Eles incluem a fala limpa, a mistura ruidosa e as saídas dos métodos STFT e Wavelet para uma condição representativa. Esses arquivos são úteis para apresentação, mas não substituem uma avaliação perceptual formal.

10 Fechamento acadêmico do protótipo integrado

10.1 Do baseline STFT ao benchmark de literatura

A investigação começou com métodos clássicos, interpretáveis e reproduzíveis. A subtração espectral STFT causal adaptativa tornou-se o primeiro baseline operacional por blocos; o Wiener causal foi mantido como alternativa menos agressiva. A DWT limiarizada mostrou ganhos pequenos, e a busca Wavelet mais ampla identificou configurações WPT em quadros objetivamente competitivos. Uma WPT causal posterior passou em gates instrumentais, mas foi rejeitada na escuta pré-ponte contra o baseline STFT. Esse resultado impediu a promoção de uma configuração apenas porque suas métricas de tonalidade eram favoráveis.

Em seguida, foi criado um harness comum para sistemas da literatura, com registro de versão, licença, taxa nativa, framing, causalidade, atraso por impulso, RTF, memória e hashes das misturas. Foram avaliados OM-LSA/IMCRA, RNNoise, WebRTC APM Noise Suppression e DeepFilterNet3, sempre com o baseline STFT como referência. O WebRTC APM foi rejeitado na configuração oficial testada por regressões de SNR e SI-SDR; o DeepFilterNet3 foi rejeitado pelo custo de memória e pela dominância do RNNoise na maior parte dos eixos. RNNoise e OM-LSA/IMCRA formaram a fronteira final pública.

Tabela 13: Resultado agregado no split público final operacional. Os candidatos foram congelados antes da abertura desse split.

Sistema	Δ SNR (dB)	Δ SI-SDR (dB)	Δ STOI	Envelope	Papel
STFT causal	3,7631	2,6476	0,00005	0,94220	Referência obrigatória
RNNoise	6,5684	5,5728	0,01817	0,83602	Finalista de maior supressão
OM-LSA/IMCRA	0,9349	0,6896	-0,00130	0,94767	Finalista de maior preservação

A avaliação privada cega foi executada antes da ponte e do endpoint, com a mesma entrada, o mesmo corte e sem normalização ou *fade*. O ranking foi RNNoise, OM-LSA/IMCRA e baseline STFT. O RNNoise recebeu notas máximas de inteligibilidade e naturalidade e não apresentou artefatos perceptíveis na tomada avaliada. Assim, foi aprovado como candidato principal para integração incremental; OM-LSA/IMCRA permaneceu como reserva. Essa aprovação é perceptual e pré-ponte: não implica continuidade temporal do endpoint nem promoção a produto.

10.2 Arquitetura implementada

A arquitetura separa o algoritmo, o transporte e a exposição do dispositivo. O DSP permanece em espaço de usuário, enquanto o driver virtual se limita a receber blocos PCM e disponibilizar o endpoint de captura.

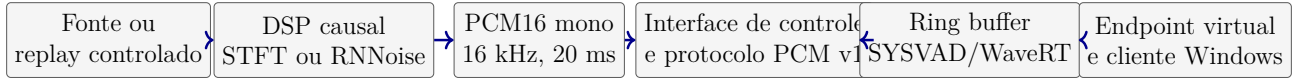


Figura 1: Arquitetura do protótipo integrado. As medições distinguem fonte, DSP, transporte, escrita no driver e consumo do endpoint.

O contrato PCM v1 usa blocos de 320 amostras, 640 bytes, sequência monotônica e uma fila não paginada com capacidade de 50 blocos. A política de *underrun* preenche a região requisitada com zeros; a política de *overrun* rejeita o bloco novo. Um único produtor possui a interface por vez. O processo de usuário captura ou recebe o áudio, processa o bloco, converte para PCM16 e envia por IOCTL apenas quando a profundidade da fila está abaixo do alvo. A interface de controle exibe estado, nível, blocos processados, perdas e latência estimada, sem armazenar áudio.

10.3 Gates isolados e prova funcional

O RNNoise persistente foi integrado por DLL no mesmo processo, com um estado contínuo, buffers fixos e resampling FIR causal $16 \rightarrow 48 \rightarrow 16$ kHz. O atraso por impulso foi de 341 amostras a 16 kHz, ou 21,3125 ms. No host, um ensaio equivalente a 600 s processou 30.000 blocos com p99 de 1,951 ms, pior caso de 18,249 ms e zero bloco acima de 20 ms. No self-test isolado da VM, 3.000 blocos tiveram p99 de 2,021 ms, pior caso de 10,691 ms e zero estouro.

O transporte host-convidado foi então separado da entrada virtualizada. Em quatro pernas pareadas, cada perna entregou exatamente 1.000 blocos, com zero erro de sequência, CRC, framing, perda ou duplicação. O RNNoise permaneceu abaixo de 20 ms por bloco. Esses resultados demonstram que o custo do DSP e a integridade do canal externo não são o bloqueador atual.

A ponte usuário-driver também foi comprovada com sinal sintético: um tom de 440 Hz foi escrito no SYSVAD e capturado por cliente externo em WAV mono, PCM16, 16 kHz e 12 s. A frequência dominante, a duração e a cauda de silêncio após a parada do produtor foram confirmadas. Portanto, o endpoint virtual é uma prova funcional de integração.

Tabela 14: Resumo dos gates aceitos antes da análise temporal do endpoint.

Gate	Evidência principal	Limite
RNNoise no host	30.000 blocos; p99 1,951 ms; pior 18,249 ms; zero estouro.	Não inclui driver ou endpoint.
RNNoise isolado na VM	3.000 blocos; p99 2,021 ms; pior 10,691 ms; zero estouro.	Não mede continuidade do endpoint.
Transporte host-convidado	1.000 blocos por perna; zero erro de sequência, CRC, framing ou perda.	Canal de ensaio, não captura física.
Ponte PCM v1 e SYSVAD	Tom de 440 Hz capturado com formato e duração corretos.	Prova funcional, não temporal prolongada.

10.4 Diagnóstico da cadência física e consumo WaveRT

Quando a ponte passou a alimentar o endpoint em tempo cadenciado, surgiram descartes e *underruns* depois do DSP. A instrumentação separou recepção, processamento, fila local, IOCTL, fila do driver, captura WASAPI e scheduler. O escritor foi melhorado com *yield*, e a barreira de partida reduziu a corrida inicial, mas nenhuma combinação de polling, temporizador, spin, afinidade, prioridade, duas a quatro vCPUs, antecipação de envio ou captura orientada a evento eliminou as perdas de forma reproduzível.

O trace interno do captador excluiu bloqueio sustentado em `GetNextPacketSize`, `GetBuffer`, cópia, análise, I/O de trace e `ReleaseBuffer`. Alguns atrasos foram localizados dentro de `SwitchToThread`; substituí-lo por spin aumentou a contenção e piorou o sistema. No modo orientado a evento, não houve falha da API nem timeout, mas retornos sinalizados chegaram a 220,532 ms. A limitação permaneceu associada ao agendamento do convidado e ao consumo WaveRT no VirtualBox/NEM.

10.5 Contrafactual SYSVAD versus HDA

O ensaio final comparou o SYSVAD modificado com o endpoint HDA emulado, na mesma VM, no mesmo boot e em ordem ABBA. Ambos usaram WASAPI compartilhado orientado a evento, sem produtor, DSP, replay ou áudio persistido. Um sinal foi considerado tardio quando o intervalo excedeu $\max(30 \text{ ms}, 2,5 T_{\text{ref}})$, em que T_{ref} é o período do dispositivo.

Tabela 15: Contrafactual de notificações orientadas a evento na mesma VM.

Perna	Sinais tardios	Períodos perdidos equivalentes	Máximo normalizado
SYSVAD A	21	381	230,21 períodos
HDA A	22	1.121	743,64 períodos
HDA B	23	248	82,18 períodos
SYSVAD B	48	304	43,72 períodos

Os máximos absolutos foram 2.302,083 ms, 7.554,401 ms, 834,825 ms e 437,176 ms, respectivamente. Dos 69 sinais tardios do SYSVAD, 54 coincidiram com um *gap* do scheduler; no

HDA, foram 37 de 45. A classificação predefinida foi `virtualbox_event_timing_supported`.

O HDA não foi um controle temporal limpo. A irregularidade é fortemente compatível com limitação global do VirtualBox/NEM ou do agendamento do convidado, e não apenas com o caminho `SYSVAD/PortCls` modificado. Entretanto, o resultado não prova que o driver `SYSVAD` esteja correto: ambos os endpoints tiveram descontinuidades, e parte dos atrasos não coincidiu com o probe geral de scheduler.

10.6 Demonstração acadêmica em três camadas

A demonstração deve evitar transformar resultados independentes em uma alegação ponta a ponta não sustentada:

1. **DSP audível:** comparação A/B entre fala ruidosa e a mesma fala tratada pelo RNNoise, usando material público ou autorizado e informando que o ensaio ocorre antes da ponte;
2. **microfone virtual funcional:** sinal sintético conhecido alimentando o `SYSVAD` e sendo capturado por um cliente Windows;
3. **integração técnica:** logs, hashes e contadores provando processamento, transporte PCM e escrita no driver sem erros de protocolo.

Não se deve apresentar uma captura longa na VM como prova de tempo real ponta a ponta, nem mascarar lacunas com repetição de amostras, aumento oportunista de filas ou edição de áudio.

10.7 Limitações metodológicas

- A escuta privada usou uma tomada autorizada e não substitui avaliação perceptual com mais participantes e material.
- O benchmark público cobre condições controladas e não representa todos os ruídos, vozes, dispositivos ou salas.
- Os resultados de custo em PC e VM não podem ser transferidos diretamente para hardware embarcado.
- A latência algorítmica foi medida ou estimada por componentes; a latência física ponta a ponta não foi comprovada.
- O endpoint virtual foi validado funcionalmente, mas não passou um gate de continuidade prolongada na VM.
- O contrafactual reforça uma limitação ambiental, mas não certifica o driver.
- Binários, datasets e áudio privado permanecem fora do repositório; a reprodução integral exige obter dependências em suas fontes oficiais.

10.8 Protocolo futuro em Windows nativo

A validação seguinte deve ocorrer em computador secundário ou SSD descartável, com Windows 11 Pro limpo, imagem restaurável, chave do BitLocker preservada, WinRE disponível

e Secure Boot desabilitado apenas na bancada. O pacote experimental exige TESTSIGNING e certificado de teste instalado em LocalMachine\Root e LocalMachine\TrustedPublisher. O thumbprint registrado é:

7ABED3D56ECAFD8B95C7B98451237673A53F899B

Um driver de teste pode causar tela azul ou impedir a inicialização; por isso, a bancada deve ser isolada e recuperável. A sequência futura é:

1. medir baseline HDA ou microfone físico;
2. instalar o SYSVAD de forma controlada;
3. validar tom determinístico, PCM v1 e captura do endpoint;
4. executar matriz pareada bypass/RNNoise;
5. exigir zero perda e zero *underrun* em repetições prolongadas;
6. medir latência física;
7. somente então usar fala controlada e escuta cega.

A migração não exige reimplementar a arquitetura. Devem ser reutilizados sem alteração inicial o RNNoise persistente, o framing de 20 ms, o protocolo PCM v1, a ponte usuário-driver, o ring buffer, a política de *underrun*, a UI e os analisadores. As adaptações concentram-se em recompilar e assinar o SYSVAD para a bancada, instalar e reverter o pacote, enumerar dispositivos, selecionar o backend de captura e substituir a automação do VirtualBox por execução local.

Para preservar a validade da comparação, a primeira rodada nativa deve manter polling, profundidades de fila, política de descarte e demais parâmetros congelados na VM. Se o baseline físico e o SYSVAD forem contínuos, a hipótese de limitação do VirtualBox/NEM ganha força. Se apenas o SYSVAD falhar, a investigação deve se concentrar em WaveRT/PortCls, timer ou callback, consumo do ring buffer e captura do endpoint. Se baseline físico e SYSVAD falharem, o foco passa a scheduler, DPC/ISR, energia, backend e drivers do host. Somente após essa classificação é permitido recalibrar filas ou períodos, sempre em ensaios pareados.

11 Avaliação do andamento e justificativa dos desvios

O cronograma anterior previa continuidade de uma revisão sistemática ampla sobre redução de ruído em sinais unidimensionais. O desvio principal foi a interrupção dessa abrangência para reformular o projeto em torno de voz humana. Esse desvio é justificado pelo feedback recebido e pela necessidade de tornar o anteprojeto mais defensável, mensurável e conectado a uma implementação realista.

A mudança exigiu reescrever objetivos, público-alvo, questionário, referências e cronograma. Em compensação, reduziu riscos relevantes: evitou uma revisão apenas descritiva, permitiu comparação sob condições controladas e produziu critérios objetivos para decidir a integração. A execução avançou além do planejamento inicial, incluindo benchmark de literatura, avaliação perceptual, RNNoise persistente, transporte cadenciado, ponte PCM v1, SYSVAD e instrumentação temporal.

O principal desvio técnico posterior foi a substituição da portabilidade embarcada imediata pela construção de um microfone virtual no Windows. Essa escolha respondeu melhor ao objetivo de disponibilizar o áudio tratado a aplicativos externos e permitiu validar interfaces, filas e comportamento do endpoint. O limite encontrado no VirtualBox/NEM foi mantido como resultado negativo relevante, em vez de ser ocultado por buffers ou edição de áudio. O andamento é considerado adequado para um protótipo acadêmico integrado, com a validação nativa e a medição física de latência explicitamente transferidas para trabalho futuro.

12 Próximos passos técnicos

O fechamento acadêmico encerra a investigação temporal exploratória na VM. Os próximos passos devem ser executados apenas quando houver ambiente apropriado:

1. preservar RNNoise como candidato principal e OM-LSA/IMCRA como reserva, sem reabrir varreduras post-hoc;
2. ampliar a avaliação perceptual com material público ou autorizado, mais ouvintes e protocolo cego;
3. reproduzir os benchmarks públicos em outro computador a partir dos scripts, manifestos e versões registradas;
4. preparar uma bancada Windows nativa isolada, recuperável e separada do computador principal;
5. portar o ambiente por reutilização do núcleo existente, limitando as adaptações iniciais a build, assinatura, instalação, enumeração e automação local;
6. executar baseline físico antes da instalação do SYSVAD;
7. repetir primeiro a configuração congelada da VM, sem retuning, para localizar o gargalo por comparação;
8. validar transporte e endpoint com sinal determinístico, exigindo zero perda e zero *underrun*;
9. medir latência física ponta a ponta por loopback ou método equivalente;
10. somente depois liberar matriz de fala controlada bypass/RNNoise e escuta cega do endpoint;
11. tratar Raspberry Pi e outras plataformas embarcadas como continuidade posterior, depois de consolidar o comportamento nativo no Windows.

13 Questionário extensionista refinado

13.1 Público-alvo e plano de aplicação

O público-alvo passa a ser composto por pessoas afetadas por ruído em comunicação por voz: estudantes em aulas remotas, profissionais em reuniões online, usuários de chamadas de voz, pessoas que gravam áudio, trabalhadores em ambientes ruidosos, participantes de jogos online, docentes, pesquisadores e profissionais de engenharia ou tecnologia interessados em processamento local de voz.

O formulário será voluntário, sem coleta obrigatória de nome, documento, endereço, voz gravada ou dado sensível. As respostas serão analisadas de forma agregada e usadas para orientar escolhas técnicas: ruídos de teste, latência-alvo, agressividade de supressão, importância de operação offline, privacidade, custo, consumo energético e acessibilidade. O questionário não substitui o benchmark experimental; ele traduz percepção de usuários em requisitos de projeto.

13.2 Questionário proposto

Texto de abertura do formulário

Este questionário integra a atividade extensionista da disciplina PTC3527 – Anteprojeto de Formatura em Telecomunicações, da Escola Politécnica da USP. O projeto investiga técnicas de redução de ruído local para voz humana em cenários de comunicação, com foco inicial em comparação técnica entre métodos baseados em STFT/Fourier e Wavelet. A participação é voluntária. Não é necessário informar nome, documento, endereço ou enviar gravações de voz. As respostas serão analisadas de forma agregada e usadas para orientar requisitos técnicos do projeto.

Bloco 1 – Contexto de uso

P1. Em quais situações o ruído mais prejudica sua comunicação por voz?

Múltipla seleção.

- | | |
|--|---|
| ☐ Reuniões online | ☐ Jogos online |
| ☐ Chamadas de voz | ☐ Gravações de áudio |
| ☐ Aulas remotas | ☐ Ambiente de trabalho |
| ☐ Transporte público | ☐ Outra: _____ |
| ☐ Ambientes industriais | |

Uso esperado no projeto: definir cenários prioritários de teste e exemplos de aplicação a serem discutidos no relatório.

P2. Quais tipos de ruído mais incomodam ou prejudicam a inteligibilidade da voz?

Múltipla seleção.

- | | |
|--|--|
| ☐ Ventilador ou ar-condicionado | ☐ Vento |
| ☐ Trânsito | ☐ Motor |
| ☐ Vozes ao fundo | ☐ Eco ou reverberação |
| ☐ Teclado | ☐ Ruído branco |
| ☐ Televisão ou música | ☐ Outro: _____ |

Uso esperado no projeto: priorizar ruídos em bases públicas, misturas sintéticas e gráficos comparativos.

P3. Em qual aplicação a melhoria da voz seria mais útil para você?

Escolha única.

- ☐ Comunicação ao vivo, como chamadas e reuniões
- ☐ Gravação posterior, como vídeos, podcasts ou aulas
- ☐ Reconhecimento automático de fala ou legendas
- ☐ Monitoramento em ambiente profissional ou industrial
- ☐ Outra: _____

Uso esperado no projeto: ajustar a ênfase entre latência, qualidade final e inteligibilidade para sistemas automáticos.

Bloco 2 – Requisitos técnicos

P4. Quais características você considera mais importantes em uma solução de redução de ruído de voz?

Escolha até três opções.

- | | |
|---|---|
| ☐ Baixa latência | ☐ Privacidade |
| ☐ Voz natural | ☐ Compatibilidade com dispositivos simples |
| ☐ Máxima remoção de ruído | ☐ Facilidade de uso |
| ☐ Funcionamento offline | ☐ Estabilidade |
| ☐ Baixo consumo de bateria | |

Uso esperado no projeto: definir pesos qualitativos para comparar qualidade, custo computacional, privacidade e viabilidade embarcada.

P5. Qual atraso máximo você considera aceitável em uma comunicação por voz ao vivo?

Escolha única.

- ☐ Imperceptível
- ☐ Até 50 ms
- ☐ Até 100 ms
- ☐ Mais de 100 ms
- ☐ Não sei avaliar

Uso esperado no projeto: orientar tamanho de janelas, buffers, salto da STFT, tamanho de blocos Wavelet e meta de tempo real.

P6. Você aceitaria pequena perda de naturalidade da voz em troca de maior remoção de ruído?

Escala de 1 a 5.

Não aceitaria

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

Aceitaria totalmente

Uso esperado no projeto: calibrar agressividade de subtração espectral, ganho espectral e limiares Wavelet.

P7. O que seria pior em uma solução de redução de ruído?

Escolha única.

- ☐ Deixar ruído demais
- ☐ Deixar a voz artificial ou metálica
- ☐ Cortar partes da fala
- ☐ Gerar atraso perceptível
- ☐ Funcionar bem só em alguns ambientes

Uso esperado no projeto: identificar qual tipo de erro deve receber maior atenção na análise qualitativa e nos ajustes de parâmetros.

Bloco 3 – Privacidade, acesso e energia

P8. Você considera importante que a solução funcione sem internet?

Escala de 1 a 5.

Pouco importante

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

Muito importante

Uso esperado no projeto: justificar processamento local e avaliar a relevância de evitar dependência de nuvem.

P9. Você se sentiria confortável enviando sua voz para processamento em nuvem?

Escolha única.

- ☐ Sim
☐ Talvez, dependendo da aplicação
☐ Não

Uso esperado no projeto: orientar a discussão sobre privacidade, processamento local e ausência de coleta de voz no projeto.

P10. Quais fatores poderiam dificultar o uso de uma solução desse tipo?

Múltipla seleção.

- | | |
|--|---|
| ☐ Hardware caro | ☐ Interface complexa |
| ☐ Necessidade de internet | ☐ Falta de compatibilidade |
| ☐ Configuração complicada | ☐ Falta de documentação |
| ☐ Alto consumo de bateria | ☐ Outro: _____ |

Uso esperado no projeto: informar escolhas de hardware, simplicidade do pipeline e critérios de acessibilidade.

P11. O consumo de energia deveria influenciar a escolha do método de redução de ruído?

Escala de 1 a 5.

Pouco importante ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5 Muito importante

Uso esperado no projeto: relacionar qualidade de áudio com custo computacional, uso futuro em dispositivos portáteis e análise de viabilidade embarcada.

Bloco 4 – Pergunta aberta

P12. Qual problema relacionado a ruído em voz você considera mais importante resolver?

Resposta aberta.

Uso esperado no projeto: revelar cenários não previstos e orientar a seleção final de ruídos, exemplos e limitações discutidas no relatório.

14 Plano indicativo de continuidade

O fechamento acadêmico encerra a busca exploratória na VM. A tabela 16 registra uma sequência futura condicionada à disponibilidade de uma bancada Windows nativa, isolada e recuperável. Ela não representa resultado já obtido nem compromisso de produto; cada etapa só deve avançar depois de cumprir o gate anterior.

Tabela 16: Sequência indicativa para validação nativa e publicação.

Etapa	Atividades	Gate ou entrega esperada
Preparação	Selecionar computador secundário ou SSD descartável; preservar BitLocker; criar imagem restaurável; registrar versões, hashes e certificado de teste.	Bancada recuperável, sem uso do computador principal e com procedimento de reversão ensaiado.
Baseline	Medir HDA ou microfone físico com tom determinístico e captura WASAPI orientada a evento, antes de instalar o SYSVAD.	Distribuição de períodos, sinais tardios e latência do ambiente nativo documentada.
Endpoint	Instalar o SYSVAD de forma controlada; validar IOCTLs, PCM v1, tom de 440 Hz, silêncio após parada e captura por cliente externo.	Prova funcional reproduzível, sem tela azul, corrupção de estado ou dependência de áudio privado.
Continuidade	Repetir matriz pareada bypass/RNNoise em execuções prolongadas, com sequência monotônica e telemetria de fila, perdas e <i>underruns</i> .	Zero perda, zero <i>underrun</i> e ausência de pausas anômalas nas repetições predefinidas.
Latência física	Medir o percurso ponta a ponta com loopback ou instrumentação externa, separando atraso algorítmico, de driver e de dispositivo.	Distribuição de latência física com p50, p95, p99 e pior caso, incluindo incerteza de medição.
Publicação	Somente após os gates técnicos, repetir fala controlada e escuta cega; publicar tabelas, manifests, scripts e limitações, sem áudio privado nem binários de terceiros.	Pacote público reproduzível, auditado e compatível com as licenças das dependências.

15 Riscos técnicos e estratégias de mitigação

Tabela 17: Riscos técnicos do projeto e mitigação.

Risco	Impacto	Mitigação
Escopo voltar a ficar amplo	Perda de foco e dificuldade de conclusão.	Manter voz humana como único tipo de sinal e STFT/Wavelet como comparação central.
Ruídos escolhidos não representarem uso real	Baixa validade externa.	Usar questionário para priorizar ruídos e combinar bases públicas com misturas controladas.
Métrica objetiva não refletir percepção	Conclusões frágeis sobre qualidade.	Combinar SNR, MSE, SI-SDR, espectrogramas, escuta crítica e, quando possível, STOI/PESQ com ressalvas.
STFT ou Wavelet exigir parâmetros demais	Risco de comparação injusta.	Separar conjunto de validação para ajuste e conjunto final para comparação. Registrar todos os parâmetros.
Latência incompatível com tempo real	Dificuldade de continuidade embarcada.	Separar latência algorítmica, latência de driver e latência de dispositivo; repetir testes com saída de menor atraso que Bluetooth e limitar janelas/blocos a valores coerentes com comunicação por voz.
Hardware inflar o projeto	Atraso e promessa excessiva.	Tratar hardware como continuidade: Raspberry Pi primeiro; ESP32 somente se o método final for simples; Arduino Uno R3 apenas como limitação futura.
Subconjunto ambiental ainda ser pequeno ou correlacionado	Validade externa limitada e risco de superestimar desempenho em poucos cenários.	Ampliar ambientes e canais DEMAND, usar segmentos separados para validação/teste e manter os ruídos sintéticos apenas como controle.
Conjunto final operacional não ser historicamente cego	Risco de influência indireta da exploração anterior.	Repetir a confirmação futura com ambientes, canais ou trechos ainda não inspecionados.
Estimativa de ruído depender de silêncio inicial	Comparação otimista para STFT e pouco realista em fala contínua.	Testar cenários sem silêncio inicial garantido e estimadores de ruído adaptativos.
Jitter do processo causal em Python	Blocos isolados podem exceder o orçamento mesmo com RTF médio baixo.	Registrar p95, p99, pior caso e blocos acima do orçamento; repetir teste Windows prolongado e considerar implementação otimizada na fase de portabilidade.
Wavelet ficar subajustada	Conclusão injusta contra a abordagem Wavelet.	Testar famílias, níveis e limiares alternativos antes de concluir inadequação do método.

Tabela 18: Análise preliminar de viabilidade embarcada.

Plataforma	Recursos	Portabilidade	Recomendação
PC	Memória e processamento abundantes; bibliotecas maduras.	Ambiente de referência em Python.	Manter como base de validação, geração de figuras e comparação objetiva.
Raspberry Pi	Centenas de MB a vários GB de RAM, Linux, USB áudio e I2S.	Python/C++ viável, com boa depuração.	Primeira plataforma embarcada plausível para teste de tempo real.
ESP32/ESP32-S3	SRAM na ordem de centenas de KB e flash em MB; I2S disponível em placas adequadas.	Exige C/C++, buffers pequenos e simplificação de dependências.	Avaliar apenas depois de reduzir memória, latência e complexidade.
Arduino Uno R3	2 KB de SRAM, 32 KB de flash, sem FPU e sem I2S nativo.	FFT/DWT para voz exige buffers e coeficientes grandes demais.	Impraticável para STFT/Wavelet de voz em tempo real sem simplificações severas ou hardware externo.

16 Considerações finais

O projeto evoluiu de uma comparação inicial entre STFT e Wavelet para um protótipo acadêmico integrado, com benchmark público, avaliação perceptual, processamento causal, transporte por blocos e microfone virtual no Windows. Essa evolução não apagou os resultados negativos: a DWT permaneceu limitada, a WPT causal foi rejeitada perceptualmente, o WebRTC APM e o DeepFilterNet3 não foram selecionados nesta bateria, e o baseline STFT revelou ruído metalizado na escuta.

O RNNoise foi aprovado como candidato principal porque combinou supressão objetiva forte, custo compatível com blocos de 20 ms e preferência na escuta cega pré-ponte. O processamento persistente passou no host e na VM; o transporte host-convidado preservou 1.000 blocos por perna sem erros de protocolo; e a ponte PCM v1 alimentou um endpoint SYSVAD funcional. Essas evidências permitem afirmar que o algoritmo, o transporte e a integração básica funcionam.

O limite decisivo apareceu depois do DSP. O consumo WaveRT e a captura do endpoint no VirtualBox/NEM apresentaram atrasos esparsos, descartes e *underruns*. O contrafactual com HDA mostrou que um endpoint não modificado também sofreu notificações tardias repetidas. O resultado reforça uma limitação global do ambiente virtualizado, mas não prova a correção do driver SYSVAD. Por isso, a escuta ponta a ponta baseada em capturas com lacunas permaneceu bloqueada.

A contribuição acadêmica principal é a separação metodológica entre qualidade do DSP,

integração funcional e continuidade temporal. A demonstração final deve refletir essa separação, combinando um A/B audível pré-ponte, um sinal sintético no microfone virtual e evidências técnicas de transporte e escrita no driver. Não há base para declarar produto pronto, distribuição segura, zero perda ou baixa latência física ponta a ponta.

O próximo passo tecnicamente responsável é uma validação em Windows nativo, em bancada isolada e recuperável. Ela deve começar por baseline físico, instalação controlada do SYSVAD e sinal determinístico; exigir zero perda e zero *underrun*; medir latência física; e somente então avançar para fala controlada e escuta cega. Até esse gate, o resultado final permanece: **protótipo acadêmico integrado, funcional em suas camadas isoladas e limitado temporalmente no ambiente virtualizado.**

Referências

- [1] OPPENHEIM, A. V.; SCHAFER, R. W. *Discrete-time signal processing*. 3. ed. Upper Saddle River: Pearson, 2010.
- [2] PROAKIS, J. G.; MANOLAKIS, D. G. *Digital signal processing: principles, algorithms, and applications*. 4. ed. Upper Saddle River: Pearson/Prentice Hall, 2007.
- [3] BOLL, S. F. Suppression of acoustic noise in speech using spectral subtraction. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, v. ASSP-27, n. 2, p. 113–120, 1979. DOI: <https://doi.org/10.1109/TASSP.1979.1163209>.
- [4] EPHRAIM, Y.; MALAH, D. Speech enhancement using a minimum mean-square error short-time spectral amplitude estimator. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, v. ASSP-32, n. 6, p. 1109–1121, 1984. DOI: <https://doi.org/10.1109/TASSP.1984.1164453>.
- [5] LOIZOU, P. C. *Speech enhancement: theory and practice*. 2. ed. Boca Raton: CRC Press, 2013.
- [6] MORETTIN, P. A. *Ondas e ondaletas: da análise de Fourier à análise de ondaletas de séries temporais*. 2. ed. São Paulo: EDUSP, 2014.
- [7] MALLAT, S. *A wavelet tour of signal processing: the sparse way*. 3. ed. Amsterdam: Academic Press, 2009.
- [8] DAUBECHIES, I. *Ten lectures on wavelets*. Philadelphia: Society for Industrial and Applied Mathematics, 1992.
- [9] DONOHO, D. L. De-noising by soft-thresholding. *IEEE Transactions on Information Theory*, v. 41, n. 3, p. 613–627, 1995. DOI: <https://doi.org/10.1109/18.382009>.
- [10] DONOHO, D. L.; JOHNSTONE, I. M. Ideal spatial adaptation by wavelet shrinkage. *Biometrika*, v. 81, n. 3, p. 425–455, 1994. DOI: <https://doi.org/10.1093/biomet/81.3.425>.
- [11] TAAL, C. H.; HENDRIKS, R. C.; HEUSDENS, R.; JENSEN, J. An algorithm for intelligibility prediction of time-frequency weighted noisy speech. *IEEE Transactions on Audio, Speech, and Language Processing*, v. 19, n. 7, p. 2125–2136, 2011. DOI: <https://doi.org/10.1109/TASL.2011.2114881>.
- [12] INTERNATIONAL TELECOMMUNICATION UNION. *ITU-T Recommendation P.862: Perceptual evaluation of speech quality (PESQ): an objective method for end-to-end speech quality assessment of narrow-band telephone networks and speech codecs*. Geneva: ITU, 2001.
- [13] VALENTINI-BOTINHAO, C. *Noisy speech database for training speech enhancement algorithms and TTS models*. University of Edinburgh DataShare, 2017. DOI: <https://doi.org/10.7488/ds/2117>.
- [14] THIEMANN, J.; ITO, N.; VINCENT, E. The Diverse Environments Multi-channel Acoustic Noise Database (DEMAND): a database of multichannel environmental noise recordings. *Proceedings of Meetings on Acoustics*, v. 19, 035081, 2013. DOI: <https://doi.org/10.1121/1.4799597>.

- [15] PANAYOTOV, V.; CHEN, G.; POVEY, D.; KHUDANPUR, S. Librispeech: an ASR corpus based on public domain audio books. In: *IEEE International Conference on Acoustics, Speech and Signal Processing*. Brisbane: IEEE, 2015. p. 5206–5210. DOI: <https://doi.org/10.1109/ICASSP.2015.7178964>.
- [16] JACKSON, Z. et al. *Free Spoken Digit Dataset*. GitHub repository. Disponível em: <https://github.com/Jakobovski/free-spoken-digit-dataset>. Acesso em: 6 jun. 2026.
- [17] RETHAGE, D.; PONS, J.; SERRA, X. A WaveNet for speech denoising. In: *IEEE International Conference on Acoustics, Speech and Signal Processing*. Calgary: IEEE, 2018. p. 5069–5073. DOI: <https://doi.org/10.1109/ICASSP.2018.8462417>.
- [18] VALIN, J.-M. A hybrid DSP/deep learning approach to real-time full-band speech enhancement. In: *IEEE Multimedia Signal Processing Workshop*. Vancouver: IEEE, 2018. DOI: <https://doi.org/10.1109/MMSP.2018.8547084>.
- [19] COHEN, I. Noise spectrum estimation in adverse environments: improved minima controlled recursive averaging. *IEEE Transactions on Speech and Audio Processing*, v. 11, n. 5, p. 466–475, 2003. DOI: <https://doi.org/10.1109/TSA.2003.811544>.
- [20] MICROSOFT. *SYSVAD Virtual Audio Device Driver Sample*. Windows Driver Samples. Disponível em: <https://learn.microsoft.com/en-us/samples/microsoft/windows-driver-samples/sysvad-virtual-audio-device-driver-sample/>. Acesso em: 14 jun. 2026.
- [21] HARRIS, C. R. et al. Array programming with NumPy. *Nature*, v. 585, p. 357–362, 2020. DOI: <https://doi.org/10.1038/s41586-020-2649-2>.
- [22] VIRTANEN, P. et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, v. 17, p. 261–272, 2020. DOI: <https://doi.org/10.1038/s41592-019-0686-2>.
- [23] LEE, G. R.; GOMMERS, R.; WASELEWSKI, F.; WOHLFAHRT, K.; O’LEARY, A. PyWavelets: A Python package for wavelet analysis. *Journal of Open Source Software*, v. 4, n. 36, p. 1237, 2019. DOI: <https://doi.org/10.21105/joss.01237>.
- [24] HUNTER, J. D. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, v. 9, n. 3, p. 90–95, 2007. DOI: <https://doi.org/10.1109/MCSE.2007.55>.